Glass Heroes Business Manager v0.3.4 RC2 Diagnostic Experimental

October 24, 2026. DIAGNOSTIC-ONLY — OWNER REVIEW PENDING. GTA testing: NOT RUN. Windows execution: NOT RUN. Crash cause and responsible script: UNKNOWN / UNATTRIBUTED. No crash correction is implemented or claimed. This is the owner-authorized RC2 development recovery, in the historical Issue #1 crash-investigation context; it does not begin v0.3.5.

Recovered checkpoint and branch

- Dedicated branch: diagnostic/v0.3.4-rc2-mechanic-transitions

- Owner-verified RC1 source commit: e312d8c9500f4a990eb993b82e8e76b55711d3de

- Verified RC1 source SHA256: 70bc95bcf0d16cbf36b61870bff7e21193c05479b79a4d8beeb45a0b734c2120

- Branch parent: 75e3623f608a05d1a2b59d12c243dbcdfc061358, the RC1 approval/package documentation commit. Its source checksum is identical to the specified RC1 checkpoint.

- RC2 instrumentation commit: 3b2ef09a96dda9f6d0325f08d29bc986167abe68

- RC2 source SHA256: 3475f93b67d5feb8d90ef30550b8a474642aeb95c77e76e8f0438ac21f30cdc9

The package's FINAL_HANDOFF.md/JSON records the exact final branch tip including its subsequent documentation/package-tool commit. Original RC1, feature/shvdn3-340-inventory-procurement and main are not modified. The original workspace checkouts are preserved. No merge or deployment is performed.

Investigation and limits

The owner reports an October 24, 2026 0xC00000FD stack overflow while Mechanic 1 transported a customer car into a service bay. This temporal context motivates instrumentation; it does not identify the responsible script, native, CLR method or mod. No October 24 crash dump, complete managed/native crash stack or owner runtime log was supplied or examined in this recovery task. Earlier October 22 observations must not be treated as the same crash.

The reviewed RC1 path goes from assignment and walking to vehicle entry, optional reverse-out, BeginVehicleDrive, approach/parking, exit and repair. Return and paid-recovery paths are included because they share transfer helpers. A conservative lexical audit parsed 419 script methods and found 86 reachable from the transfer roots, with no direct cycle among those unqualified script calls. Qualified calls such as data.Save(), SDK/native implementations, CLR internals, delegates, runtime reentry and other scripts are outside that audit. It is not a semantic C# analyzer or evidence that this script is crash-free. See INVESTIGATION.json and verification/diagnostic340/investigate.py.

No task retry, recursion guard that skips gameplay, native replacement, vehicle movement correction or AI rewrite is introduced. The trace is designed to provide context for a future real dump and owner reproduction. A fatal stack overflow may terminate the process before finally blocks or any logger can run; instrumentation cannot catch or repair it.

Exact source-level scope

Only experimental/shvdn3/Install/scripts/GlassHeroesBusiness.3.cs changes in the gameplay source. The original v2 sources and both generations' project/solution files are unchanged. The v3 project still directly links the matching installation source; target net48, language C# 7.3, official

ScriptHookVDotNet3 NuGet version 3.6.0.

Ten existing methods receive one diagnostic Begin call and a try/finally End scope:

1. UpdateOneCustomerOrder — per-order state before existing validation and state-machine work.
2. AssignCustomerMechanic — assignment, quote reservation/save and handoff selection context.
3. BeginVehicleWalk — original walk-route setup/task dispatch.
4. BeginEnterVehicle — original clear/enter-vehicle dispatch.
5. StartValetDeparture — original optional reverse-out or drive dispatch.
6. BeginVehicleDrive — original ability/aggressiveness/drive task dispatch.
7. UpdateVehicleApproach — original approach, park dispatch and bounded drive retry.
8. UpdateVehicleParking — original park/exit and bounded parking retry.
9. FailVehicleTransfer — existing failure/cancellation or paid-return recovery decision.
10. ProcessPaidReturnRecovery — existing safe empty-car recovery.

Their original statements, branches, return behavior, exception propagation and native ordering are byte-identical after undoing the mechanical indentation introduced by the scopes. SOURCE_EXCERPTS.md provides before/after examples, and source.diff is the complete unified source diff. Applying that diff to the pinned RC1 reproduces the RC2 source byte-for-byte. Body reindentation accounts for much of the displayed diff; it is not a gameplay rewrite.

New diagnostic-only members are CustomerVehicleTraceState, CustomerVehicleTrace, CaptureCustomerVehicleTrace, BeginCustomerVehicleTrace, EndCustomerVehicleTrace and AppendCustomerVehicleDiagnosticLine, plus one instance trace field and one static safe logger field. They capture managed order/stage/slot/handle/mode/route-index/retry information without querying entity existence, position, speed, models, tasks or the world. Entity Handle properties identify existing wrappers; diagnostic code never validates or manipulates those entities.

Bounded logging and performance

Diagnostics begin only after the existing boot-activation gate. No constructor native calls, new Tick callback, per-frame poll, delay/yield, background thread or stack-trace capture is added. Calls run where the existing state machine already operates, normally at the existing one-second customer refresh. Existing explicit/manual recovery calls keep their prior scheduling.

There are 64 fixed snapshot slots. Stable operation/order/state at the same or lower nesting depth produces no extra log formatting, logging-clock read or I/O. A greater nesting depth can produce a bounded breadcrumb, which may help identify runtime reentry. Nesting can also be normal: assignment → walking, reverse fallback → drive, and failure → recovery are examples. An observed depth alone does not prove recursion or a fault.

At most 12 admitted ENTER records per ten-second logging window, each with a corresponding attempted EXIT. Failed ENTER writes also spend the budget. EXIT attempts for already admitted scopes can complete after the originating window. Rapid state changes are coalesced/suppressed; this is not a complete flight recorder. Successful later ENTER reports the count of budget-suppressed candidates. Diagnostic time is UTC wall-clock only for logging; no business clock uses it.

The existing unchanged DiagnosticLog supplies reentry protection, contained writer failure and a 30-second write-failure backoff. The trace adds its own writer reentry guard, contains its own errors and always balances admitted/suppressed scopes through finally on ordinary returns/exceptions. A diagnostic failure must not replace an existing gameplay exception or stop native task dispatch.

Only scripts/GlassHeroesBusiness/CustomerVehicle.RC2.log and CustomerVehicle.RC2.log.previous rotate. Each file is bounded to 128 KiB by UTF-8 size; records are capped at 2048 characters. One previous file is retained. Thus diagnostic footprint is approximately 256 KiB across two files. Existing Errors.log and source-identity logging remain unchanged. See DIAGNOSTIC_INSTRUCTIONS.md for pairing and limitations.

Save, financial and gameplay compatibility

No Save.ini or Settings.ini key, default, schema, migration, serialized field, stored coordinate or waypoint mask changes. No existing owner data/settings were opened or modified. The file-rotation regression uses synthetic temporary sentinel files only. RC2 adds no user setting to enable/disable its always-on bounded diagnostic scopes; rollback to the exact RC1 source removes them.

The exact-restoration guard proves that removing only the new diagnostic block and ten scopes restores the entire verified RC1 source. It also checks 426 existing members remain untouched, project/v2 sources remain byte-identical, and all nine inherited fixture edits are only no-op diagnostic sink adapters with every original assertion preserved. This protects vehicle/AI behavior, native task ordering, saved route geometry, invoices/quotes, stock, manager procurement, receipts, business saves and active-unpaused game-time progression. The original Tick, startup defaults, controller arbitration and all financial/stock/save methods outside the scopes remain untouched.

Unchanged source logic is not a substitute for native/runtime QA. Synchronous bounded trace writes add some overhead and can affect timing; no engine performance measurement has been run. The owner should compare behavior/performance with the exact RC1 checkpoint using copied data.

Actual compilation and regression results

Independent RC1 recovery run: 954/954 inherited behavior/navigation checks PASS; actual Release build PASS, 0 errors / two existing warnings. RC1 source hash matched the owner value before any instrumentation.

Final RC2: 985/985 behavior/navigation PASS, including all inherited 954 and 31 new trace cases. Navigation remains 212/212. Static checks: 21/21 PASS. Inventory structure: 13/13 PASS. Diagnostic-only exact-restoration/fixture preservation: PASS. Real clean Release compilation: 0 errors, inherited warnings CS0169 and CS0649. Build tool SDK 8.0.426; product net48/C# 7.3; SHVDN3 package 3.6.0.

The new cases extract real production trace/state/safe-writer members and five instrumented transfer methods. They exercise stable outer/nested updates, three orders, stage/retry changes, flood bounds/recovery, clock/write failures, writer reentry, nested depth, boot/null references, native ordering, early return, cancellation, original exception propagation, reverse/park/exit/retry and real temporary file rotation. Inherited accounting fixtures use no-op trace adapters; new trace tests exercise the real diagnostics separately.

Tests run on deterministic .NET 8 adapters, not GTA natives. They do not prove compatibility with GTA, SHVDN Enhanced, controller hardware, the owner's MLO/heavy mod stack or CLR crash behavior. No crash reproduction, dump capture or successful gameplay session was performed by Codex. GTA testing NOT RUN. Windows execution NOT RUN. Enhanced runtime compatibility NOT VERIFIED.

Evidence is in baseline/ and final-validation/, including commands.json, actual compiler/test output, checksums and preservation reports. The final suite ran before committing the instrumented worktree: its source_revision records the then-current RC1 documentation HEAD, while source_sha256 records the actual RC2 bytes compiled/tested. Those bytes were subsequently committed as 3b2ef09a96dda9f6d0325f08d29bc986167abe68; report/package generation checks exact equality to that commit. Reproduce in the full Git checkout with Python 3/.NET 8 tooling (cloud shell: source /workspace/.cloud-setup/env.sh):

```
python verification/inventory340/validate.py --output /tmp/gh-rc2-review --issue36-integration
 --integration-approved --diagnostic-rc2
python verification/diagnostic340/investigate.py
```

The approval flag records the already accepted Issue #36 integration in RC1; it does not accept RC2 gameplay. The full preservation audit requires the Git history/protected baseline files. Product rebuild from the package requires only its matching source/project and NuGet references:

```
dotnet build VisualStudio/GlassHeroesBusiness.V3.Experimental.csproj -c Release
```

Delivery and owner review

The review ZIP contains one installation script, matching Visual Studio project/solution, manifest/checksums, all new documentation and Word/PDF guide, full source diff, test sources and raw evidence. It contains no business Save.ini/Settings.ini or runtime DLL. Source retains RC1's v0.3.4 DEV header/startup wording so the original source outside diagnostics is unchanged; RC2 identity comes from this diagnostic-only package, source SHA and RC2 VEHICLE log prefix.

The separate archive receipt records independent ZIP extraction, every manifest hash and another clean product compilation. Code/regression results apply to its byte-identical source; documentation-only commits do not change gameplay qualification. Finished status is ready for owner review, not an invented stability certification or crash fix.

Follow INSTALL.md, GTA_TEST_CHECKLIST.md and ROLLBACK.md only during a later owner-run test. No live installation, save rewrite, protected-branch merge, GitHub public release or v0.3.5 work is performed by this task.

RC2 diagnostic capture and interpretation

Diagnostic-only; GTA testing NOT RUN; crash cause UNATTRIBUTED. Keep the exact RC2 source hash, complete pre-test business-data backup, game build, ScriptHookV/SHVDN runtime versions and mod list with each test. Use an isolated complete copy of business data. Do not alter working routes/AI to chase labels or force a reproduction.

Files

After the existing startup delay and first eligible operation, the script attempts to create scripts/GlassHeroesBusiness/CustomerVehicle.RC2.log. The previous 128 KiB segment is CustomerVehicle.RC2.log.previous. Copy both promptly after a test or crash and before reloading; they rotate automatically. No new trace file before an eligible operation is normal. Storage failure can suppress the trace while gameplay continues.

Also preserve the existing Glass Heroes Errors.log/source-identity records, ScriptHookVDotNet and ScriptHookV logs, launcher/Windows crash events and any crash/minidump produced by the current host. Preserve pre/post-test complete business-data copies together with those logs. Do not send the only live Save.ini without keeping a backup.

Record fields

The existing safe writer prepends UTC time. RC2 VEHICLE marks this diagnostic candidate; session changes on a new trace instance. ENTER and EXIT share seq within that session. ENTER names the operation. mechanic=1 means slot 0; mechanic=0 means unassigned. order is the existing order ID; vehicle/worker are existing managed entity handles, which can be reused by GTA and must be correlated with session/order/time. stage is the existing CustomerStage name.

mode is a bit mask: 1 paid, 2 physical vehicle handoff, 4 automatic lot, 8 vehicle teleport, 16 vehicle transfers enabled, 32 reverse-out enabled. walk is existing zero-based current node/count. driveRetries and parkingRetries are existing counters. phase is the mechanic's existing arrival phase (0 none, 1 checkpoint, 2 break-bound, 3 at break, 4 bay-bound, 5 working, 6 returning, 7 blocked). depth is diagnostic scope nesting, not a measured CLR stack depth. suppressed counts candidates dropped by the trace-entry rate limit since its last successful ENTER.

Expected transfer sequence can include assignment/walking, BeginEnterVehicle, StartValetDeparture, BeginVehicleDrive, UpdateVehicleApproach and UpdateVehicleParking. The existing stage names distinguish bay intake from customer pickup return. Original optional reverse, custom/automatic parking, retries and recovery decisions still apply.

What a breadcrumb can establish

A matching EXIT means the scope's finally reached its logging attempt. It can occur on a normal return, early return or an ordinary propagated exception; it does not certify success. Consult the unchanged Errors.log for exceptions. An unmatched ENTER can indicate interruption, fatal failure, unavailable storage or missing/rotated output. It does not prove that operation caused a crash. There is no per-native trace and no complete event history.

Repeated/increasing depth may provide reentry context but ordinary nested helpers also produce depth greater than one. Stable snapshots are deliberately silent. Rate limiting, coalescing, writer backoff, file rotation and catastrophic process termination can omit relevant records. Absence of a breadcrumb cannot exclude this script or implicate another one. No logger is guaranteed to execute when the CLR has exhausted its stack.

For the next failure, record local time with timezone, elapsed session time, mechanic/order/stage if visible, exact player action and native/custom/automatic drive modes. Retain any host-produced dump. A dump with managed/native stack frames and module/runtime identity is needed to attribute 0xC00000FD; the last visible driving operation alone is insufficient. This package does not configure dump capture or modify Windows/GTA runtime settings automatically.

RC2 Diagnostic Experimental — owner testing installation reference

For later owner-run testing only. No installation was performed by Codex. Diagnostic-only; GTA testing NOT RUN; no claim of a crash fix. Do not copy the complete archive into GTA.

1. Close GTA and every script host. Back up the exact installed script plus the COMPLETE scripts/GlassHeroesBusiness directory outside every script loading path. Include Save.ini, Settings.ini, CSVs, profiles and companion files. Preserve the verified RC1 source and its matching full backup separately.

2. Use a disposable complete data copy for the first RC2 test. Do not alternate source versions against the sole live business save or mix finance/receipt journals from different backup sets.

3. Ensure exactly one Glass Heroes Business Manager script is active. Keep older .2.cs/.3.cs variants and compiled Business Manager DLLs outside all script loading directories. Leave unrelated scripts untouched.

4. Copy only Install/scripts/GlassHeroesBusiness.3.cs from this package to the existing GTA/scripts/GlassHeroesBusiness.3.cs path. No business data/settings file or runtime DLL is included; do not replace the data directory.

5. Verify SHA256 with Get-FileHash on Windows. Expected RC2 source: 3475f93b67d5feb8d90ef30550b8a474642aeb95c77e76e8f0438ac21f30cdc9. Confirm matching SOURCE_SHA256.txt/PACKAGE_MANIFEST.json and source-identity log. RC1's unchanged DEV startup

label is not a different candidate; use the hash and RC2 VEHICLE diagnostics to identify this build.

6. Retain the owner's current game/ScriptHookV/SHVDN/mod configuration. The RC1 documentation names GTA V Legacy 1.0.3889.0 and SHVDN Enhanced 3.9.0.6; actual RC2 runtime compatibility is NOT VERIFIED. Compilation uses the official SHVDN3 3.6.0 API and net48/C#7.3. Do not update runtime dependencies or mod settings as an implicit crash fix.

7. Preserve normal startup delay and follow GTA_TEST_CHECKLIST.md. Copy both RC2 diagnostic files and all host/error logs before reloading after a failure. No owner data is changed by package extraction or build; running GTA normally can save legitimate business transactions.

RC2 GTA checklist — ALL NOT RUN by Codex

Owner review pending. Diagnostic-only. Use copied business data and the exact package hash; record every outcome, timestamp/timezone, game/SHVDN/mod versions and reproduction sequence. Do not certify a crash fix from a clean session.

1. Cold launch and script reload with the usual heavy mod stack. Verify unchanged startup delay, one Business Manager instance, RC2 source identity and no pre-activation world work or saved-data rewrite.

2. Verify copied funds, inventory, incoming shipments, receipts/invoices, employees, active promises, Save.ini/Settings.ini values, waypoints/masks and manager settings against RC1 before testing. Open/close menus with keyboard/controller/mixed input; preserve prior appearance and navigation.

3. Reproduce the reported Mechanic 1 customer-car transfer under the SAME existing bay/route/vehicle settings. Observe walking, entry, optional reverse, driving, approach, parking, exit and repair. Capture local crash time and elapsed session time if it fails. Do not move bay markers or reset routes to force success.

4. Repeat with other mechanics and concurrent orders. Check custom/automatic lot and existing reverse settings on disposable data only; retain original tested settings for comparison. Verify the trace identifies each order/slot and normal nested operations without per-second stable-state spam.

5. Let a repair complete and return the vehicle through the normal pickup path. Verify the original single invoice, parts consumption, staff release and customer departure. Exercise existing bounded obstruction/parking retry and paid-return recovery where safely reproducible; no new movement/recovery behavior is expected.

6. Walk/drive outside the streaming range and return; allow existing manager takeover/reload paths. Verify no duplicate vehicle, invoice, stock, receipt or restored order. Save/reload the complete copied business and compare finances and active promises.

7. Pause/resume, remain in menus and close/reopen GTA. Supplier ETA, wages, manager/remote work and all business progression must advance only during active unpaused game time; no offline catch-up. Confirm existing physical wholesale player collection/GPS and receipts still work.

8. Observe frame behavior, script responsiveness and diagnostic growth during extended idle and repeated transfers. Stable state should be silent; rapid changes can be rate limited. A missing diagnostic line is not proof that an operation was not executed. Do not force disk failures or change permissions in the live installation to test backoff.

9. After a crash, preserve both CustomerVehicle.RC2.log files, existing Errors.log, SHVDN/ScriptHookV/host/Windows crash records, any dump and exact pre/post-test full data copies BEFORE another reload. Record source/package hashes and all mod/runtime versions. Keep the cause UNATTRIBUTED until stack evidence identifies it.

10. For rollback, close GTA and follow ROLLBACK.md. Preserve failed-test evidence. Report any behavioral/data/performance difference from RC1 separately from crash attribution.

RC2 rollback and data preservation

Close GTA and all script hosts. Preserve the complete RC2 test data directory and both RC2 diagnostic log segments, existing error/host logs and dumps outside script loading paths. Keep the original RC2 source/package hashes with this evidence.

Remove the RC2 script from active loading paths and restore the verified RC1 script plus its COMPLETE matching pre-test business-data/settings backup. Owner-specified RC1 source commit: e312d8c9500f4a990eb993b82e8e76b55711d3de; source SHA256: 70bc95bcf0d16cbf36b61870bff7e21193c05479b79a4d8beeb45a0b734c2120. The approval-only RC1 tip has the same source. Ensure exactly one generation/source/DLL is active before relaunch.

Do not reset, hand-edit or combine the sole live Save.ini with another version's finance/receipt/profile files. Restore all companion files together. RC2 adds no save/config schema or migration, but normal gameplay transactions can legitimately change data during testing. Keep both pre/post-test copies rather than discarding evidence. Diagnostic files may remain archived separately; no tool in this package restores or rewrites live files automatically.

v0.3.4 RC2 Diagnostic Experimental — October 24, 2026

- Recovered and checksum-verified owner RC1 source; dedicated RC2 diagnostic branch preserves RC1/main.

- Added bounded managed entry/finally-exit breadcrumbs around ten mechanic/customer-vehicle transfer scopes. Stable-state coalescing, 64 snapshot slots, 12 ENTER admissions per ten seconds, writer reentry/failure containment and one 128 KiB rotated backup.

- Original vehicle tasks, AI, routes, invoice/stock/save logic and active-unpaused progression unchanged. No new saved key, settings default or migration. Diagnostic-only; no verified crash correction.

- Actual Release: 0 errors / two inherited warnings. Full inherited 954 plus 31 new trace tests: 985/985 PASS. Static 21/21, inventory structure 13/13 and exact-restoration preservation PASS. GTA/Windows execution NOT RUN; cause UNATTRIBUTED.

- Packaged matching source/project, checksums/manifest, raw validation, source diff, Word/PDF guide, capture instructions, GTA checklist and full-data rollback reference for owner review. No merge, deployment or v0.3.5 work.

RC2 before/after source excerpts

Full exact unified diff: source.diff. Gameplay bodies are preserved inside new diagnostic scopes.

BeginEnterVehicle

Before (verified RC1)

```
    private void BeginEnterVehicle(CustomerOrder order, bool returning)
    {
        Ped worker = AssignedWorker(order);
        if (worker == null || order.Vehicle == null || !order.Vehicle.Exists())
        { FailVehicleTransfer(order, !returning, "Mechanic or customer vehicle missing");
return; }
        GTA.Native.Function.Call(GTA.Native.Hash.CLEAR_PED_TASKS, worker.Handle);
        GTA.Native.Function.Call(GTA.Native.Hash.TASK_ENTER_VEHICLE,
            worker.Handle, order.Vehicle.Handle, 18000, -1, 1.0f, 1, 0);
        order.Stage = returning ? CustomerStage.MechanicEnteringForReturn :
CustomerStage.MechanicEnteringVehicle;
        order.SecondsInStage = 0;
    }
```

After (diagnostic RC2)

```
    private void BeginEnterVehicle(CustomerOrder order, bool returning)
    {
        long trace = BeginCustomerVehicleTrace(order, 3);
        try
        {
            Ped worker = AssignedWorker(order);
            if (worker == null || order.Vehicle == null || !order.Vehicle.Exists())
            { FailVehicleTransfer(order, !returning, "Mechanic or customer vehicle missing");
return; }
            GTA.Native.Function.Call(GTA.Native.Hash.CLEAR_PED_TASKS, worker.Handle);
            GTA.Native.Function.Call(GTA.Native.Hash.TASK_ENTER_VEHICLE,
                worker.Handle, order.Vehicle.Handle, 18000, -1, 1.0f, 1, 0);
            order.Stage = returning ? CustomerStage.MechanicEnteringForReturn :
CustomerStage.MechanicEnteringVehicle;
            order.SecondsInStage = 0;
        }
        finally { EndCustomerVehicleTrace(order, trace); }
    }
```

BeginVehicleDrive

Before (verified RC1)

```
    private void BeginVehicleDrive(CustomerOrder order, bool toBay)
    {
        Ped worker = AssignedWorker(order);
        if (worker == null) { FailVehicleTransfer(order, toBay, "Mechanic disappeared while
driving"); return; }
        WorkBay destination = VehicleDestination(order, toBay);
        order.ParkingRetries = 0;
        float speed = order.AutomaticLot ? 3.0f : _vehicleTransferSpeed;
        float stopRange = order.AutomaticLot ? 3.0f : 6.0f;
        // Best available native driver skill is set in BOTH parking modes;
        // it does not add navmesh or guarantee collisions are avoided.
        GTA.Native.Function.Call(GTA.Native.Hash.SET_DRIVER_ABILITY, worker.Handle, 1.0f);
        GTA.Native.Function.Call(GTA.Native.Hash.SET_DRIVER_AGGRESSIVENESS, worker.Handle,
0.0f);
        if (order.AutomaticLot)
        {
            if (toBay && IsAutoLotParkingBlocked(order))
            {
                // Do not start a task into an already occupied car space.
                CancelCustomerOrder("Auto bay occupied; change side/offset or use MLO mode",
true);
                return;
            }
        }
        GTA.Native.Function.Call(GTA.Native.Hash.TASK_VEHICLE_DRIVE_TO_COORD_LONGRANGE,
            worker.Handle, order.Vehicle.Handle, destination.Position.X,
destination.Position.Y,
            destination.Position.Z, speed, CarefulDrivingStyle, stopRange);
        order.Stage = toBay ? CustomerStage.MechanicDrivingToBay :
CustomerStage.MechanicDrivingToDropoff;
        order.SecondsInStage = 0;
        order.LastDrivePosition = order.Vehicle.Position;
        order.DriveNoProgressSeconds = 0;
        order.DriveRetryCount = 0;
        Notify(_mechanics[order.MechanicSlot].Name + (toBay ? " driving car into service." : "
 returning car to drop-off."));
    }
```

After (diagnostic RC2)

```
    private void BeginVehicleDrive(CustomerOrder order, bool toBay)
    {
        long trace = BeginCustomerVehicleTrace(order, 5);
        try
        {
            Ped worker = AssignedWorker(order);
            if (worker == null) { FailVehicleTransfer(order, toBay, "Mechanic disappeared
while driving"); return; }
            WorkBay destination = VehicleDestination(order, toBay);
            order.ParkingRetries = 0;
```

```
            float speed = order.AutomaticLot ? 3.0f : _vehicleTransferSpeed;
            float stopRange = order.AutomaticLot ? 3.0f : 6.0f;
            // Best available native driver skill is set in BOTH parking modes;
            // it does not add navmesh or guarantee collisions are avoided.
            GTA.Native.Function.Call(GTA.Native.Hash.SET_DRIVER_ABILITY, worker.Handle, 1.0f);
            GTA.Native.Function.Call(GTA.Native.Hash.SET_DRIVER_AGGRESSIVENESS, worker.Handle,
 0.0f);
            if (order.AutomaticLot)
            {
                if (toBay && IsAutoLotParkingBlocked(order))
                {
                    // Do not start a task into an already occupied car space.
                    CancelCustomerOrder("Auto bay occupied; change side/offset or use MLO
mode", true);
                    return;
                }
            }
            GTA.Native.Function.Call(GTA.Native.Hash.TASK_VEHICLE_DRIVE_TO_COORD_LONGRANGE,
                worker.Handle, order.Vehicle.Handle, destination.Position.X,
destination.Position.Y,
                destination.Position.Z, speed, CarefulDrivingStyle, stopRange);
            order.Stage = toBay ? CustomerStage.MechanicDrivingToBay :
CustomerStage.MechanicDrivingToDropoff;
            order.SecondsInStage = 0;
            order.LastDrivePosition = order.Vehicle.Position;
            order.DriveNoProgressSeconds = 0;
            order.DriveRetryCount = 0;
            Notify(_mechanics[order.MechanicSlot].Name + (toBay ? " driving car into service."
 : " returning car to drop-off."));
        }
        finally { EndCustomerVehicleTrace(order, trace); }
    }
```